

Informe del Proyecto - Taller GraphQL

Facultad Barberi de Ingeniería, Diseño y Ciencias Aplicadas
Departamento de Computación y Sistemas Inteligentes
Computación en Internet III

Github: <https://github.com/FranciscoLozano0/tallerGraphQL.git>

despliegue: <https://tallergraphql-ks5t.onrender.com>

Integrantes

Santiago Lopez - A00401318

Francisco Lozano - A00401813

Manuel Rojas - A00402557

Haider Valencia - A00400766

1. Descripción de la Funcionalidad

Tomando como base la aplicación backend "Alacena" (originalmente desarrollada bajo una arquitectura de API REST para la gestión alimentaria), se realizó la migración exitosa a una API **GraphQL** utilizando Node.js, NestJS y TypeScript. Este sistema mantiene las funcionalidades clave, optimizando la obtención y mutación de datos.

- **Gestión de Usuarios (Módulo Base):** Se implementó un sistema con roles de usuario (superadmin, user). El usuario **superadmin** tiene los privilegios para crear, visualizar, modificar y eliminar cualquier registro de usuarios (CRUD completo). Los usuarios autenticados user pueden consultar otros usuarios según los requerimientos, pero tienen restringida cualquier mutación (modificación o eliminación) a terceros.
- **Gestión del Inventario (Módulo 1 - Entidad Principal):** En este proyecto, se seleccionó al "Inventario de Alimentos" como proyecto/entidad principal. Los usuarios autenticados tienen operaciones de creación, lectura, actualización y borrado (CRUD) restringidas exclusivamente a **sus propios ítems de inventario**. Por otra parte, los administradores pueden gestionar el inventario global de cualquier usuario.
- **Relaciones con otros Módulos:** Los ítems del "Inventario" (Módulo 1) están intrínsecamente vinculados con "Usuarios" y "Productos" (Módulo 2), evidenciando la interconexión con Graph y resolución de dependencias por medio de campos anidados.

2. Configuración y Ejecución

Requisitos Previos

- **Node.js:** v18 o superior.
- **Base de Datos:** PostgreSQL disponible en el servidor local o mediante docker-compose.

Pasos

1. **Clonar el proyecto** desde el repositorio de GitHub.
2. **Instalar dependencias:**

```
npm install
```

```
# o alternativamente, si usa yarn o bun
```

```
bun install
```

3. **Configurar el entorno:** Renombrar el archivo de variables locales `.env.template` a `.env` (si aplica) con las métricas de la Base de datos y Secretos de JWT.

4. **Ejecutar en modo Desarrollo:**

```
npm run start:dev
```

El *Playground* de Apollo/GraphQL estará disponible en <http://localhost:3000/graphql> (o el endpoint configurado).

3. Endpoints (Consultas/Mutaciones) y Tipos de Documentos

Toda interacción de datos se centraliza en el endpoint `/graphql`.

Principales Consultas (Queries):

- `users`: Obtener el listado de usuarios (con validación de rol administrativo o autenticado).
- `user(id)`: Visualizar un usuario concreto.
- `inventoryItems`: Obtiene los objetos del proyecto (inventario) correspondientes al usuario.
- `inventoryItem(id)`: Detalles de un elemento.
- *Adicionales*: `products`, `recipes`, `nutritionProfile`.

Principales Mutaciones (Mutations):

- `register(input)`, `login(input)`: Métodos base para Auth.
- `createUser(input)`, `updateUser(id, input)`, `removeUser(id)`: Manejables por superadmin.
- `createInventoryItem(input)`, `updateInventoryItem(id, input)`, `removeInventoryItem(id)`: Mutaciones del Módulo 1 (verificando propiedad mediante el JWT del solicitante).

Tipo de Documentos: El uso de TypeScript fuertemente tipado en conjunto con `@nestjs/graphql` define tipos devueltos (`User`, `InventoryItem`, `Product`) e `Inputs` (`CreateUserDto`, `UpdateInventoryItemDto`). Las respuestas frente a errores se manejan uniformando un envoltorio HTTP amigable, o mediante excepciones nativas de GraphQL (`GraphQLError`).

4. Seguridad, Autenticación y Autorización

- Se instrumentó una estrategia JWT (**JSON Web Token**). Las consultas y mutaciones restringen su acceso haciendo uso de un Guard especializado (`GqlAuthGuard`), que extrae el encabezado `Authorization: Bearer <token>` del contexto extendido de Apollo Server.
- Se ha aplicado un decorador personalizado `@Roles()` para garantizar que únicamente los usuarios con las credenciales necesarias (ej. superadmin) alcancen operaciones críticas. Todos los demás tienen restricciones directas.

5. Dificultades y Elementos no Desarrollados

- **Curva de migración de la Autenticación:** Trasladar los Guards convencionales de Express a *GqlExecutionContext* tuvo algo de fricción, ya que fue necesario mutar la validación y extracción de los Headers HTTP.
- **Consultas anidadas extensas:** A nivel de optimización de rendimiento (evitar el problema de $N+1$), la migración demandó vigilar cómo extraían las relaciones como el "Producto asociado al Inventario" sin saturar la DB de múltiples *SELECTs*.
- En base a las pautas dadas, algunas mecánicas de validación profunda a sub-campos no se concluyeron por tiempo, y se basan en validaciones estándares del *class-validator*.

6. Pruebas (Supertest y Postman)

- **Postman:** Dentro del directorio del código en la carpeta postman/se ubica el archivo Alacena TallerGraphQL Completa.postman_collection.json. Contiene ejemplos estandarizados para consumir el Playground incluyendo Colecciones para la Creación de Usuarios, Login y las Mutaciones/Consultas del CRUD de Usuarios e Inventario.
- **Supertest:** Las validaciones End-to-End se conservaron y ajustaron para disparar consultas vía el método POST con el objeto en crudo {"query": "mutation { ... }"} asegurando una sólida fase de testing automático. Puede corroborarse con `npm run test:e2e`.

7. Despliegue en la Nube

El repositorio asume el despliegue automático configurado en **Render**:

1. El Backend se configura tras cada *merge/push* en el repositorio apuntando a inicializar `npm run build` e iniciar el compilado a través de `npm run start`.
2. Se vinculó a una instancia PostgreSQL alojada en *neondb*.